Plugin API

Plugins extend Quill with new commands, views and integrations. They are written in TypeScript or JavaScript and run inside the app with access to a documented API. This page explains how a plugin is structured, the main parts of the API, and how to publish a plugin.

Plugin structure

A plugin is a folder inside the vault's .quill/plugins directory. It must contain a manifest named plugin.json and a JavaScript entry file. Quill loads every enabled plugin when a vault opens.

The manifest

The manifest describes the plugin and the permissions it needs:

```
{
"id": "word-count",
"name": "Word Count",
"version": "1.4.0",
"minAppVersion": "3.0.0",
"main": "main.js",
"permissions": ["notes:read", "statusbar"]
}
```

The id must be unique and may contain only lower-case letters, digits and hyphens. minAppVersion stops the plugin from loading on older versions of Quill that lack an API it relies on. Quill refuses to load a plugin whose manifest is missing a required field and shows the reason in the plugin settings.

Permissions

Plugins must declare every permission they use. When a user enables a plugin, Quill lists its permissions and asks for confirmation. Calling an API without the matching permission throws an error.

- notes:read allows reading note content and metadata.
- notes:write allows creating, changing and deleting notes.
- network allows outgoing HTTP requests through the API's fetch function.
- statusbar allows adding items to the status bar.

Plugins have no direct access to the file system outside the vault, and they cannot load native code.

The plugin lifecycle

The entry file exports a class that extends Plugin. Quill calls onload when the plugin is enabled and onunload when it is disabled or the vault closes. Anything registered through the API during onload is cleaned up automatically on unload, so most plugins never need to write an onunload method at all.

```
import { Plugin } from "quill";

export default class WordCount extends Plugin {
async onload() {
const item = this.app.statusBar.addItem("words");
this.registerEvent(this.app.notes.onChange((note) => {
item.setText(`${note.text.split(/\s+/).length} words`);
}));
}
}
```

Main API areas

Commands

Commands appear in the command palette and can be given keyboard shortcuts by the user. Register one with this.addCommand and an object with an id, a name and a run callback. Command ids are prefixed with the plugin id automatically, so two plugins can both define a command named refresh without clashing.

```
this.addCommand({
id: "insert-timestamp",
name: "Insert timestamp",
run: (editor) => editor.replaceSelection(new Date().toISOString()),
});
```

Notes

The app.notes object reads and writes notes. get(path) returns a note with its text and front matter, create(path, text) makes a new note, and onChange fires whenever any note is saved, whether by the user, by sync or by another plugin. Writes made through the API are recorded in version history like any other edit.

Views

A plugin can add a panel to the right sidebar with registerView. Views are rendered with plain DOM elements; Quill does not require or bundle any front-end framework. Use the CSS variables exposed by the active theme so that your view matches light and dark themes.

Settings

Plugins store their settings with this.loadData() and this.saveData(obj). The data is kept as JSON in the plugin's folder and synced along with the vault, so settings follow the user to other devices.

Debugging

Open the developer console with Ctrl+Shift+I to see log output and errors from plugins. Changes to a plugin's files are not picked up automatically; use the Reload plugins command after editing. Enable QUILL_LOG_LEVEL=debug to see when each plugin loads and how long its onload took, which helps when tracking down slow startups.

Performance guidelines

Plugins run on the same thread as the editor, so slow work in an event handler makes typing feel sluggish. Keep onChange handlers fast, debounce anything that runs on every keystroke, and move heavy work to a web worker. Quill warns in the console when a single handler takes longer than 50 milliseconds.

Publishing

To list a plugin in the community directory, publish its source in a public repository with a release that contains plugin.json and main.js, then submit the repository address through the form on the Quill website. Every new plugin and every update that adds a permission is reviewed by a person before it appears in the directory. Reviews usually take about a week.

Plugins in the directory must use an open-source license, must not collect analytics without asking the user first, and must not show advertisements.

Versioning and compatibility

The plugin API follows semantic versioning. Methods are never removed in a minor release; a deprecated method keeps working for at least two minor releases and logs a warning when used. Breaking changes happen only in major versions and are announced in the release notes well in advance.